

How Middleware Lets You Customize Your Agent Harness

LangChain · Sydney Runkle · 2026-03-26 · 原文 / Source

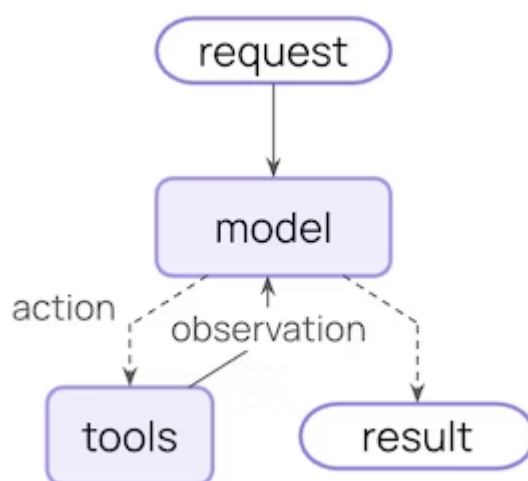
Agent harnesses are what help build an agent, they connect an LLM to its environment and let it do things.

When you're building an agent, it's likely you'll want build an application specific agent harness. "Agent Middleware" empowers you to build on top of LangChain and Deep Agent's solid foundation, but customize them for your use case.

What are agent harnesses

An agent is a system built around a model. The model needs to be connected to an environment, data, memory, and tools. Agent harnesses are the system that helps you do that.

The core of every agent harness is the same, and remarkably simple: an LLM, running in a loop, calling tools. Simple as it is, there's power in this core loop.



LangChain contains `create_agent` - an abstraction with just this core loop.

Why you would want to customize your agent harnesses

Different agent use cases have different needs. They may require different agent harnesses.

Some parts of the an agent harness - like instructions or tools - are pretty easy to customize.

`create_agent` in LangChain lets you pass in a system prompt and tools for example.

Other parts are more involved. What if you want always run a certain step before the model executes? What if you always want to check the tool output for certain things?

Things that involve changing the core loop of the agent are trickier to change. When done correctly, it enables really powerful customization that still allows you to build on the core harness.

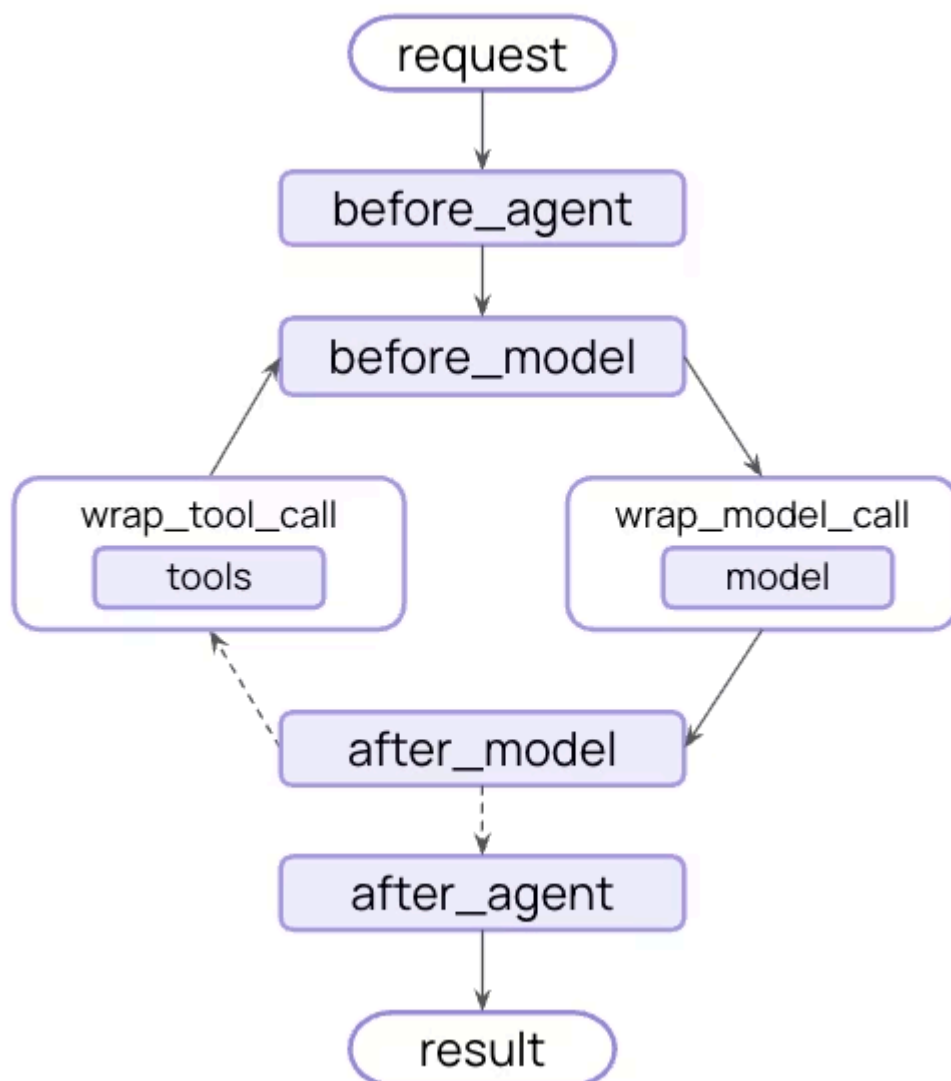
`AgentMiddleware` is our answer for this - how we let people customize LangChain agents.

What is agent middleware?



“Middleware” is a general term often used in other software engineering practices, but below we refer to a different system which we call agent middleware.

Middleware exposes a set of hooks that let you run custom logic before and after each step, so you can control what happens at every stage of the loop:



- **before_agent** : Runs once on invocation. Good for loading memory, connecting to resources, or validating initial input.
- **before_model** : Fires before each model call. Use it to trim history or catch PII before it hits the LLM.

- `wrap_model_call` : Wraps the model call end-to-end. Caching, retries, and dynamic model requests like changing available tools all live here.
- `wrap_tool_call` : Wraps tool execution similarly. Inject context, intercept results, or gate which tools actually run.
- `after_model` : Runs after the model responds but before tools execute. The most natural place for human-in-the-loop.
- `after_agent` : Runs once on completion. Save results, send notifications, clean up.

Middleware are composable, so you can mix and match to your heart's content.

LangChain ships a set of prebuilt middleware for the most common patterns, like summarization, retries, and PII redaction. Builders can also subclass the `AgentMiddleware` class to write your own for anything bespoke to your business.

Examples of Middleware

Customization needs tend to cluster around the same themes. Below are the most common use cases:

Business logic & compliance. Some things can't live in a prompt, like PII redaction and content moderation. These are deterministic policies that have to fire every time. You can't prompt your way to HIPAA compliance.

Deep dive: PII detection

LangChain's builtin `PIIMiddleware` implements `before_model` and `after_model` hooks. It has the ability to mask/redact/hash PII on model inputs, outputs, and tool outputs. It can also raise a `PIIDetectionError` for the most critical PII detection situations.

Dynamic agent control. Middleware can reshape the agent at runtime: inject tools based on current state, swap the model mid-task, update the system prompt as context evolves. It's active control over how the agent behaves at each step.

Deep dive: dynamic tool selection

LangChain's `LLMToolSelectorMiddleware` runs a fast LLM in the `wrap_model_call` hook to identify which tools from a registry are relevant for a given request. It then binds those tools to the model request to minimize context bloat from unnecessary tools in the main model call.

Context management. The model is only as good as what you put in front of it. For example, you might need to summarize when you're approaching token limits and trim noisy tool inputs/outputs. Context engineering is a runtime problem, not a one-time prompt problem.

Deep dive: summarization and context offloading

LangChain's builtin SummarizationMiddleware implements the `before_model` hook. To avoid context overflow, if message history exceeds a certain token threshold, its contents are summarized before being passed to the model. Extensions of this middleware implement a `wrap_tool_call` hook to extend verbose tool call inputs and outputs to the filesystem.

Production readiness. Middleware allows you to build in model/tool retry logic, model fallbacks, and human-in-the-loop with interrupts. These kinds of features don't show up in demos, but are essential for production agents.

Deep dive: model retries

LangChain's builtin ModelRetryMiddleware implements the `wrap_model_call` hook in order to wrap a model's API call with a retry handler. This handler supports retry configuration such as retry count, backoff factor, and initial delay (to troubleshoot rate limiting).

Toolsets. Inject tools that require custom setup and teardown around the agent loop like connecting to an external tool server, initializing a shell, or spinning up a sandbox.

Deep dive: shell tool middleware

LangChain's ShellToolMiddleware implements the `before_agent` and `after_agent` hooks in order to initialize and teardown shell resources around the core agent loop. It also adds the shell tool to the model's list of tools.

Deep Agents case study

Deep Agents is a batteries included agent harness built entirely on `create_agent`, LangChain's standard entry point for building agents, with an opinionated middleware stack on top.

Here are a few of the middlewares that power Deep Agents:

- `FilesystemMiddleware`: file-based context on/offloading and long-term memory
- `SubagentMiddleware`: subagents with context isolation
- `SummarizationMiddleware`: context overflow management for long-running tasks
- `SkillsMiddleware`: progressive disclosure of specialized capabilities
- And more!

For a full review of the middleware powering Deep Agents, see this guide and Vivek's anatomy of a harness post.

On top of all of this - you can add even more middleware to Deep Agents to customize it for your use case!

Why we're betting on agent middleware

Models are getting more capable, and that will change parts of the middleware stack. Some of what Deep Agents does today — summarization, tool selection, output trimming — will eventually be absorbed into the model itself.

But the underlying need won't change. Builders will always need levers for customization: deterministic policy enforcement, production readiness guardrails, use-case-specific business logic. None of that moves into the model. The harness is still where it lives, and middleware is still the cleanest way to expose it.

We've seen this play out since the LangChain v1 launch. Middleware lets different teams own different concerns, keeps business logic decoupled from core agent code, and makes it easy to reuse logic across an org. Building Deep Agents entirely on top of it convinced us it's the right abstraction.

Want to get started from a barebones agent harness? Try out middleware in `create_agent`.

Want to build on top of a more robust agent harness? Try out middleware in `create_deep_agent`.

Want to contribute your own middleware? See guides for that here.